COMP30220 Distributed Systems
Assignment Report

Group Name: Dysfunctional Systems

Group members	Students Numbers	E-mail Address
Hiya Garg	22204345	hiya.garg@ucdconnect.ie
Ema Florea	22748395	floreaemaelena@gmail.com
Ryan Koenig	22201444	koenigryan4@gmail.com

Git Repo: https://gitlab.com/ucd-cs-rem/comp30220-2026/dysfunctional-systems

Section 1: Proposal Submission

Introduction

The main idea of our project revolves around a Microservice-oriented application for handling restaurant and customer order information. We plan to use a system of service choreography to handle communication among microservices. The main functions of the application are to retrieve order information such as status, items, and cost; retrieve restaurant information such as menus, hours, ratings, and potentially customer loyalty programs (stretch); and finally: post orders to the service and restaurant, managing payment as necessary.

Services

Our system will have 3 microservices. We plan to use Kubernetes for container orchestration as it naturally provides scaling and auto-healing capabilities, but other alternatives may be considered based on complexity.

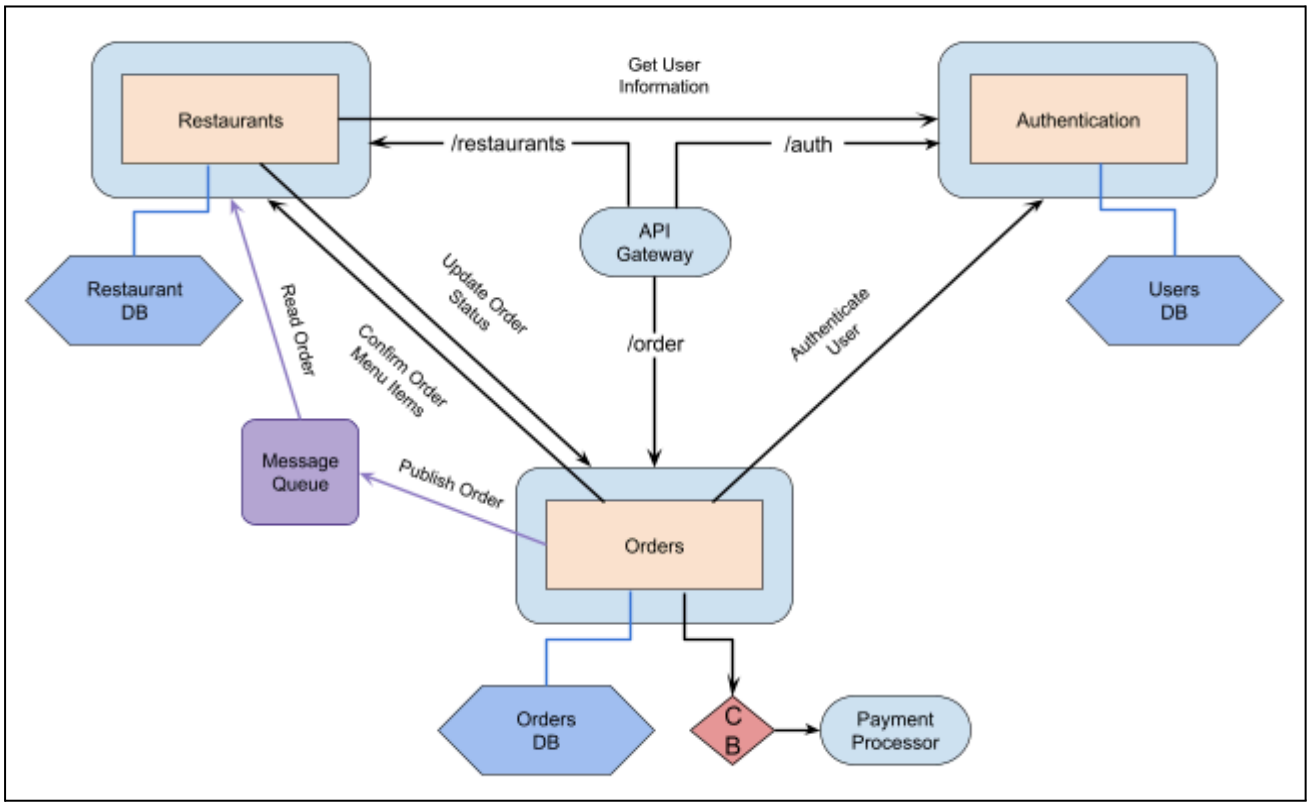
1. Orders service: manages current and past delivery orders, holding onto current order status, restaurant, items, and price in order to make it easy for customers and restaurants to quickly view information about their recent orders. This service also publishes orders to a message queue when appropriate so that a Restaurant service can receive the orders and process them. The order service also makes calls to a mock payment processor; this connection is made through a circuit breaker, ensuring that if the payment processor goes down, its shutdown and recovery is handled gracefully.

2. Restaurant service: stores and delivers restaurant information when called. The service would also be designed to handle delivery hours, menus, prices, and loyalty schemes for the restaurants, as well as handing orders placed into the queue by the Orders service.

3. User authentication and sign-up service: This service lets users register, and grants authentication tokens on valid login. Any subsequent user interactions with the system use this token to validate the user. This service also helps keep user accounts and private data such as past orders or loyalty rewards, separate.

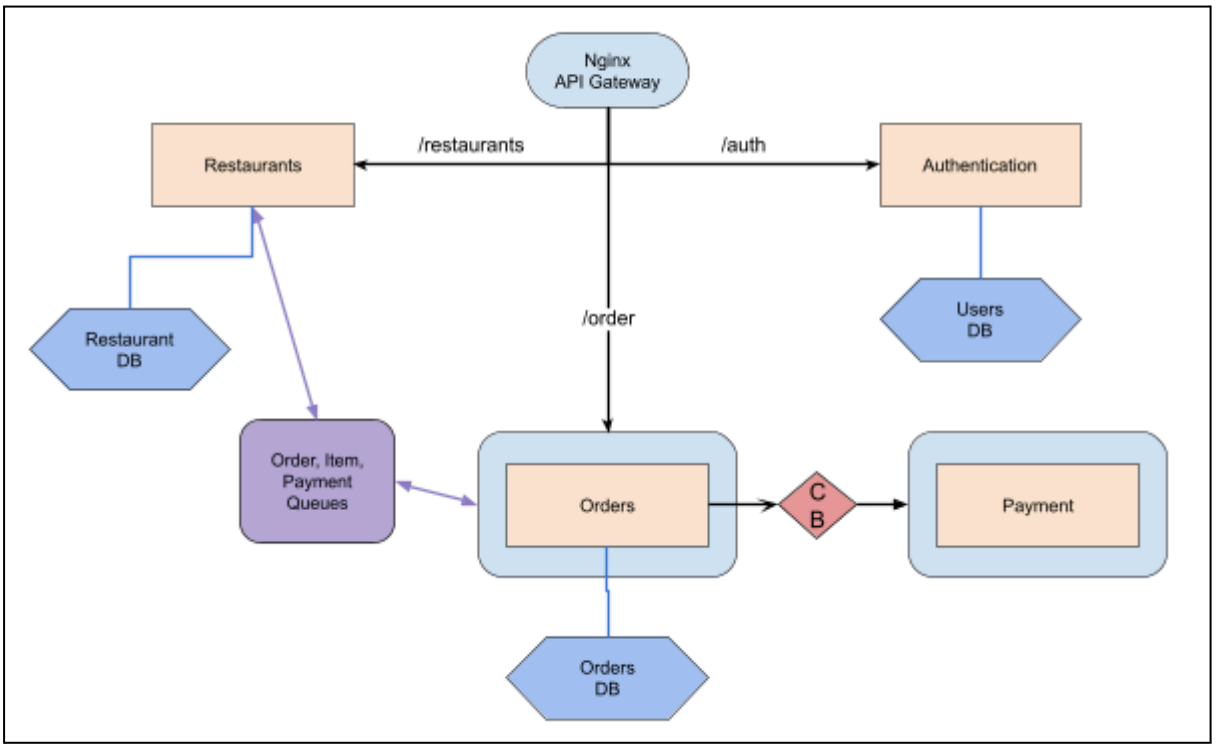
Service Connection Diagram Legend:

- Pods / services
- Horizontal scaling
- Message queue
- API Gateway
- Database
- Circuit Breaker
- ↑ - API call
- database query



Section 2: Final Architecture

The system is a distributed food ordering platform built on a microservices architecture. All requests enter through an Nginx API Gateway (port 8080) which routes traffic to one of the four services. Order and Restaurant service communicate asynchronously via RabbitMQ. No service calls another directly except for the Order-to-Payment HTTP call protected by a circuit breaker. Each service owns its own database.



Description of Services

Authentication Service

This service handles customer/user registration and login. It allows registration using name, email and password, and login using an existing email and password. Upon either action, the service issues the user a signed JWT (Json Web Token) which is used in subsequent requests to validate the user.

Stores customer data in its own SQLite database. It operates independently, and serves as an entry point to the system.

Endpoint	Method	Description
/api/authentication/register	POST	Register a new customer, returns JWT
/api/authentication/login	POST	Login with email and password, returns JWT

Notes:

1. Invalid credentials during login returns 401
2. Duplicate email registration returns 400

3. Passwords are hashed using bcrypt before storing them into the database. Plaintext is never stored.

Restaurant Service

Owns and manages the menu and inventory. It is responsible for validating that items in a newly placed order exist, that there is sufficient stock for each, and also responsible for calculating the total bill amount.

Stores menu and stock data in its own SQLite database. Communicates via a queue, RabbitMQ, with the Order Service (see the queue table for more details).

Endpoint	Method	Description
/api/restaurant/menu	GET	Returns available menu items
/api/restaurant/menu/{item_id}	GET	Return a single menu item
/api/restaurant/stock	GET	Stock/quantity available/remaining for menu items

Notes:

1. Get /menu only returns the name, description and price of items; quantity is excluded since it is not relevant to the customer.
2. When an order is placed, if any item in the order is invalid or its quantity is not sufficient, an items_unavailable event is published to the order_service which then immediately rejects (“cancels”) the order
3. When an order is canceled (by a user or due to payment failure), the release_items event is used to restore quantities of ordered items automatically.

Order Service

Owns and manages the order lifecycle. It drives each order through the full pipeline from stock reservation to payment, to confirmation or cancelation.

Stores order and order items information in its own PostgreSQL database. It communicates with the Restaurant service via RabbitMQ (see queue table for more details). It calls the Payment service directly via HTTP, wrapped in a circuit breaker to prevent cascading failures.

Endpoint	Method	Description
/api/orders/	POST	Place new order (JWT required)
/api/orders/{order_id}	GET	Retrieve a single order information (JWT required)
/api/orders/my-orders	GET	Retrieve all past orders (JWT required)
/api/orders/cancel/{order_id}	PATCH	Cancel a pending order (JWT required)
/api/orders/circuit-breaker/status	GET	Returns the current state of the circuit breaker

Notes:

1. The top four endpoints require a valid JWT. Unauthenticated requests receive a 401 with HATEOAS links to register/login (we have implemented HATEAOS for every endpoint).
2. Customers can only view and cancel their own orders.
3. Only orders in “pending” state can be canceled.

4. Payment failures are categorized as: payment_declined (402), payment_processor_failed (service error or circuit breaker is open), or unexpected_error (fallback error)

Payment Service

Mocks/simulates an external payment processor or API (for example, Stripe).

Endpoint	Method	Description
/api/payment/charge	POST	Process a payment charge
/api/payment/mock/break	POST	Force the service into a failing state (For demos). Returns 503 on all subsequent requests
/api/payment/mock/fix	POST	Restore the service to healthy state (For demos)

Maintains no persistent database. Receives HTTP requests from the Order Service only for charging payment.

Notes:

1. Every payment has a 20% probability of being declined (HTTP 402) to simulate scenarios where the user either doesn't have sufficient account balance, or provided faulty credentials. This is a legitimate business outcome, and not a system failure. It does not add to the failure count of the circuit breaker.
2. Every payment also has a simulated 10-second processing delay to make this service the primary bottleneck under load.

Queues used for Event-based Comms

Queue	Producer	Consumer
order_created	Order Service: when a customer places a new order	Restaurant Service: checks stock and reserves item (decreases quantity)
items_reserved	Restaurant Service: when order items are validated, their stock is successfully reserved, and total bill amount is returned	Order Service: Proceeds with payment
items_unavailable	Restaurant Service: when item(s) is invalid or out of stock	Order service: cancels the order
payment_success	Order Service: when payment charge is successful.	Restaurant Service: no action. Just to complete saga
release_items	Order Service: when payment fails or customer cancels order	Restaurant Service: restores reserved stock back into inventory

Technology Stack

1. Services: Python, FastAPI
2. Containerisation: Docker, Docker Compose
3. API Gateway: Nginx
4. Message Broker: RabbitMQ
5. Order Database: PostgreSQL
6. Authentication and Restaurant Databases: SQLite
7. Authentication: JWT (HS256), bcrypt for password hashing
8. Async HTTP Client: httpx
9. Async RabbitMQ Client: aio_pika
10. ORM: SQLModel

Design patterns implemented: Database-per-Service, Circuit Breaker, API Gateway, SAGA, HATEOAS

Section 3: Member Contribution

Hiya
- Implemented Order service, and queue-based coordination between services.
- Implemented the circuit breaker as a guard in front of Payment service.
- Contributed to the report.

Ryan
- Implemented Payment service to model payment behaviour with a processing delay and a failure probability.
- Implemented Restaurant service.
- Recorded video and contributed to the report.

Ema
- Implemented Authentication service for registration, login, and token-based validation.
- Implemented the autoscaler mechanism for scaling Order and Payment service containers.
- Contributed to the report.

Section 4: How are Scalability and Fault Tolerance handled?

Scalability

Scaling is handled by a custom autoscaler.py that uses the items_reserved queue length to scale both Order and Payment Services. The Payments service has a processing delay (a simulated 10s delay in this project) for each payment request, and since Orders makes a synchronous HTTP call to it, this quickly becomes a bottleneck when orders are arriving faster than their payments can be processed. Scaling the Orders Service alone would simply result in more concurrent requests piling up against a single Payments instance. Therefore, both services are scaled together.

For the same reason, the items_reserved queue is chosen because it effectively captures the pressure at the payment stage of order processing. A rising queue depth directly indicates that the orders/payment pipeline cannot keep up.

The scaler polls the RabbitMQ Management API every 5s to check items_reserved queue depth:

1. Scale up: if queue depth exceeds 3 messages and fewer than 2 instances are running, then one more replica is added.
2. Scale down: if queue depth reaches 0 and more than 1 instance is running, then it removes additional replicas.

Instances are bound between 1 (minimum) and 2 (maximum). These parameters, along with the poll interval and queue depth threshold are configurable in the autoscaler script allowing the scaling behaviour to be tuned.

The load_test.py is used to demonstrate and validate the scaling behaviour.

Fault Tolerance

A) Circuit Breaker
The Payment Service is guarded by a circuit breaker. The Order Service wraps its HTTP calls to Payment with this circuit breaker. After 3 consecutive payment failures, the circuit opens and immediately rejects further payment requests for 15s, effectively marking the service unavailable for that timeout. This prevents bombarding the broken payment service with more requests. After the timeout, the circuit transitions to half-open and allows one probe request through to test if Payment service has recovered. If payment is successfully processed, the circuit breaker closes marking the Payment service as being fully available. If not, the breaker opens for another 15 second cycle.

B) Durable Message Queues
All RabbitMQ queues are declared with durable=True and are published with DeliveryMode.PERSISTENT so messages survive broker restart, and they persist in their queues even when the consumer service is down/broken/unable to process those messages.

C) Saga Pattern and Compensating Transaction
When a new valid order is placed, the Restaurant reserves the item quantities (decreases the stock in its database), and only then Orders service attempts a payment. If the payment fails due to the payment being declined, or the payment service being down, or the customer canceling the order, the system implements a compensating transaction to undo the item reservation action. A release_items event is published to the Restaurant service that adds the reserved quantities back to the stock. This ensures that stock is never permanently locked by an order that will not be fulfilled.

Section 5: Reflection on Experience

Changes from Proposal

Kubernetes -> Docker Compose: The proposal outlined Kubernetes for container orchestration. After further research, we concluded that it would add significant complexity to our work, especially getting the message queuing and databases (both of which are stateful) set up. We primarily considered Kubernetes because it provides automated scaling and fault tolerance, but Docker compose with a custom autoscaler achieved the same scaling behaviour with far less overhead. It also allowed us to understand and implement the scaling logic ourselves rather than delegating it entirely to a platform. It encouraged us to think about what metrics to scale on, what thresholds to use, and which services to scale.

Payment Service as a separate service: The proposal described payment as a function of the Order service because at the time we were still figuring out how to implement this. In the final system,

payment is a fully independent service which mocks/simulates payment processing being handled by an external provider. It also makes the circuit breaker more useful that guards an actual service boundary rather than an internal function call.

Dropped Stretch Goals: The proposal mentioned restaurant hours, ratings, loyalty programs as potential features. However, we removed them since they didn't meaningfully contribute to the distributed systems aspect of the assignment. They are just additional business logic that increase service complexity without adding architectural interest.

Additions Not in Proposal: We added RabbitMQ queues and persistent message delivery. We also adopted HATEOAS to make the requests return contextually relevant operations for the user. Initially, we didn't plan to implement saga, but once we introduced stock reservation as a committed step before payment, it naturally necessitated implementing saga.

Our Choices

No frontend: This was a well-deliberated consensus reached by the team. Building a frontend would not have contributed to demonstrating distributed systems concepts. The system is fully interactable via API calls, and sufficient for demonstrating scaling, fault tolerance and comms. None of us had experience with frontend, and building one would only have consumed time better spent on distributed systems aspects of the project.

Action-oriented Cancel Endpoint: The cancel endpoint contains the cancel action within the URL, rather than using a pure REST endpoint for updating the order status via a request body. However, a generic update endpoint would require validating permitted order status transitions, and allows the customers' attempts to set arbitrary statuses. A dedicated action endpoint makes the action explicit and exclusive, which is more secure by design even though the URL contains the action within it.

Selection of Databases: The Authentication and Restaurant services use SQLite because they both have simple, low-contention read/write operations, and they are not scaled either, so a full database server is not justified for them. However, the Order Service requires concurrent writes from multiple scaled instances, so it is given a full-fledged PostgreSQL database. We ensured to match the database to each service's requirements rather than applying a uniform technology across all services.